

Notebook Execution Order

- To ensure the project runs correctly, you have to make sure that the notebook `23127130_23127168_23127231_23127238_23127269.ipynb` **MUST** be run first, because it performs the dataset merging and preparation steps required for the other notebooks.
- After that, the remaining notebooks can be run, but to follow the order of the Objective, you should run like this order:

1. `23127130_23127168_23127231_23127238_23127269.ipynb`
2. `23127130_23127238.ipynb`
3. `23127168.ipynb`
4. `23127231_23127269.ipynb`

YouTube Presentation

- Presentation video link: [Group 08 - Data Visualize 2](#)

I. Introduction

This notebook is used to merging the data getting from the [WDI website](#) as well as introduce, summarize, and preprocess the dataset we choose for other notebooks that perform analysis and evaluation.

II. Used libraries

This project uses several common Python libraries for data loading, processing, and visualization.

pandas

`pandas` is the main library used to work with tabular data.

In our project, it is used to:

- read `.csv` files into DataFrames
 - clean missing values such as `..`
 - reshape data from wide format to long format using `melt()`
 - merge datasets together
 - group, sort, and summarize data
 - prepare data for plotting
-

numpy

`numpy` is used for numerical operations.

In this project, it can help with:

- handling numeric arrays efficiently
- supporting missing values and mathematical operations
- creating derived calculations if needed

Even if it is not heavily used in every notebook cell, it is useful when working with numerical analysis and statistics.

os

`os` is used to work with file paths and folders in the operating system.

In our project, `os` is especially useful when:

- datasets are too large and must be downloaded in multiple parts
- several `.csv` files need to be stored in the same folder
- we want to build file paths dynamically instead of typing each file name manually
- we need to scan a folder and combine many split files into one dataset

This makes the project more flexible and easier to run when the raw data is stored in multiple parts.

seaborn

`seaborn` is a statistical data visualization library built on top of `matplotlib`.

In our project, it is used to:

- create clearer and more attractive charts
 - visualize relationships between variables
 - generate plots such as heatmaps, scatter plots, regression plots, and line plots
-

matplotlib.pyplot

`matplotlib.pyplot` is the core plotting library.

In our project, it is used to:

- create and customize figures

- control chart size, titles, labels, and layout
 - display multiple subplots in one figure
-

`matplotlib.lines.Line2D`

`Line2D` is a class from `matplotlib.lines` that helps create custom line elements.

In our project, it is useful for:

- drawing custom legend entries for dumbbell charts
- representing markers and connecting lines clearly
- improving the readability of comparisons between two values
- customizing the appearance of chart elements beyond default settings

This is especially helpful when we want to make the dumbbell chart more informative and visually clean.

Note:

Some of the libraries may not import in this notebook since the main purpose of this notebook is introducing and preparing the dataset for the analysis stage.

III. Importing libraries and building support functions

Import necessary libraries

```
In [1]: import pandas as pd
import numpy as np
import os
```

Merging the dataset into one

```
In [2]: ID_COLUMNS = ["Country Name", "Country Code", "Series Name", "Series Code"]
MERGE_KEYS = ["Country Name", "Year"]

def _read_dataset(dataset_path: str) -> pd.DataFrame:
    for encoding in ("utf-8", "cp1252", "latin1"):
        try:
            return pd.read_csv(dataset_path, encoding=encoding)
        except UnicodeDecodeError:
            pass
    return pd.read_csv(dataset_path, encoding="latin1")
```

```

def clean_and_sort_dataset(dataset_path: str) -> pd.DataFrame:
    df = _read_dataset(dataset_path)
    df = df.iloc[:-5].replace("..", pd.NA)

    year_cols = [col for col in df.columns if "YR" in col]

    df_long = df.melt(
        id_vars=ID_COLUMNS,
        value_vars=year_cols,
        var_name="Year",
        value_name="Value",
    )

    df_long["Year"] = df_long["Year"].str.extract(r"(\d{4})").astype(int)
    df_long["Value"] = pd.to_numeric(df_long["Value"], errors="coerce")
    df_long = df_long.sort_values("Year")

    df_final = (
        df_long.pivot_table(
            index=MERGE_KEYS,
            columns="Series Name",
            values="Value",
            aggfunc="mean",
        )
        .reset_index()
        .sort_values(MERGE_KEYS)
        .reset_index(drop=True)
    )

    df_final.columns.name = None
    return df_final

```

```
FOLDER_PATH = "../raw_dataset"
```

```

def get_dataset_paths(folder_path: str) -> list:
    dataset_paths = []
    for filename in os.listdir(folder_path):
        if filename.endswith(".csv"):
            dataset_paths.append(os.path.join(folder_path, filename))
    return dataset_paths

def merge_all_datasets(folder_path: str) -> pd.DataFrame:
    dataset_paths = get_dataset_paths(folder_path)
    if not dataset_paths:
        raise FileNotFoundError(f"No CSV files found in {folder_path}.")

    merged_df = clean_and_sort_dataset(dataset_paths[0])

    for dataset_path in dataset_paths[1:]:
        print(f"Processing {dataset_path}...")
        next_df = clean_and_sort_dataset(dataset_path)

        merged_df = merged_df.merge(
            next_df,

```

```

        on=MERGE_KEYS,
        how="outer",
        suffixes=("", "_new"),
    )

    overlapping_columns = [
        col for col in merged_df.columns
        if col.endswith("_new")
    ]

    for new_col in overlapping_columns:
        original_col = new_col.replace("_new", "")
        merged_df[original_col] = merged_df[original_col].combine_first(merged_df[new_col])
        merged_df = merged_df.drop(columns=new_col)

    return merged_df.sort_values(MERGE_KEYS).reset_index(drop=True)

print("Merging datasets...")
data = merge_all_datasets(FOLDER_PATH)
print("Datasets merged successfully.")

```

Merging datasets...

Processing ../raw_dataset\902bcd0c-744b-4241-95a7-e91ee25d4ae4_Series - Metadata.csv...

Datasets merged successfully.

IV. Overview about the dataset

- This dataset provides a multidimensional view of socio-economic conditions in Southeast Asia during the period **2000-2020**. It covers a group of Southeast Asian countries and brings together indicators from several major development domains, including **education, health, economic policy and debt**. By organizing these indicators in a consistent country-year structure, the dataset enables both cross-country comparison and temporal analysis across two decades of development change.
- The selected time range from **2000 to 2020** is especially meaningful because it captures a period of substantial transformation in Southeast Asia. During these years, many countries in the region experienced rapid economic growth, structural change, expansion in public services, and major policy efforts aimed at improving human development outcomes. As a result, the dataset is suitable for examining not only the level of development in each country, but also the pace and direction of change over time.

```

In [3]: SEA_COUNTRIES = [
        "Brunei Darussalam",
        "Cambodia",
        "Indonesia",
        "Lao PDR",
        "Malaysia",
        "Myanmar",
        "Philippines",

```

```

    "Singapore",
    "Thailand",
    "Timor-Leste",
    "Viet Nam",
]

# Add more here if needed
CORE_VARS = [
    # Education indicators: Question 1
    "Government expenditure on education, total (% of GDP)",
    "Government expenditure on education, total (% of government expenditure)",
    "Pupil-teacher ratio, primary",
    "Pupil-teacher ratio, secondary",
    "Trained teachers in primary education (% of total teachers)",
    "Trained teachers in secondary education (% of total teachers)",
    "Children out of school (% of primary school age)",
    "School enrollment, primary (% net)",
]

# General dataset statistics without showing the variable list
dataset_statistics = pd.DataFrame(
    {
        "Statistic": [
            "Number of rows",
            "Number of columns",
            "Number of countries",
            "Time period",
            "Min year",
            "Max year",
            "Total missing values",
            "Duplicate country-year rows",
        ],
        "Value": [
            len(data),
            data.shape[1],
            data["Country Name"].nunique(),
            f"{data['Year'].min()}-{data['Year'].max()}",
            data["Year"].min(),
            data["Year"].max(),
            int(data.isna().sum().sum()),
            int(data.duplicated(subset=["Country Name", "Year"]).sum()),
        ],
    }
)

display(dataset_statistics)

```

	Statistic	Value
0	Number of rows	231
1	Number of columns	62
2	Number of countries	11
3	Time period	2000-2020
4	Min year	2000
5	Max year	2020
6	Total missing values	4046
7	Duplicate country-year rows	0

```
In [4]: # Print all the columns in the dataset
print("Columns in the dataset:")
for col in data.columns:
    print(f"- {col}")
```

Columns in the dataset:

- Country Name
- Year
- Adolescents out of school (% of lower secondary school age)
- Children out of school (% of primary school age)
- Current account balance (% of GDP)
- Current health expenditure (% of GDP)
- Current health expenditure per capita, PPP (current international \$)
- Debt service (PPG and IMF only, % of exports of goods, services and primary income)
- Domestic general government health expenditure (% of general government expenditure)
- Educational attainment, Doctoral or equivalent, population 25+, female (%) (cumulative)
- Educational attainment, Doctoral or equivalent, population 25+, male (%) (cumulative)
- Educational attainment, Doctoral or equivalent, population 25+, total (%) (cumulative)
- Educational attainment, at least Bachelor's or equivalent, population 25+, female (%) (cumulative)
- Educational attainment, at least Bachelor's or equivalent, population 25+, male (%) (cumulative)
- Educational attainment, at least Bachelor's or equivalent, population 25+, total (%) (cumulative)
- Educational attainment, at least Master's or equivalent, population 25+, female (%) (cumulative)
- Educational attainment, at least Master's or equivalent, population 25+, male (%) (cumulative)
- Educational attainment, at least Master's or equivalent, population 25+, total (%) (cumulative)
- Educational attainment, at least completed primary, population 25+ years, total (%) (cumulative)
- Educational attainment, at least completed upper secondary, population 25+, total (%) (cumulative)
- Exports of goods and services (% of GDP)
- External debt stocks (% of GNI)
- Foreign direct investment, net inflows (% of GDP)
- Foreign direct investment, net outflows (% of GDP)
- GDP (current US\$)
- GDP growth (annual %)
- GDP per capita (constant 2015 US\$)
- GDP per capita growth (annual %)
- Government expenditure on education, total (% of GDP)
- Government expenditure on education, total (% of government expenditure)
- Gross savings (% of GDP)
- Immunization, DPT (% of children ages 12-23 months)
- Imports of goods and services (% of GDP)
- Life expectancy at birth, female (years)
- Life expectancy at birth, male (years)
- Life expectancy at birth, total (years)
- Literacy rate, youth (ages 15-24), gender parity index (GPI)
- Literacy rate, youth total (% of people ages 15-24)
- Mortality caused by road traffic injury (per 100,000 population)
- Mortality from CVD, cancer, diabetes or CRD between exact ages 30 and 70 (%)
- Mortality from CVD, cancer, diabetes or CRD between exact ages 30 and 70, female (%)

- Mortality from CVD, cancer, diabetes or CRD between exact ages 30 and 70, male (%)
- Mortality rate, infant (per 1,000 live births)
- Mortality rate, under-5 (per 1,000 live births)
- School enrollment, primary (% net)
- School enrollment, primary (gross), gender parity index (GPI)
- School enrollment, primary, female (% net)
- School enrollment, primary, male (% net)
- School enrollment, secondary (% net)
- School enrollment, secondary (gross), gender parity index (GPI)
- School enrollment, secondary, female (% net)
- School enrollment, secondary, male (% net)
- School enrollment, tertiary (% gross)
- School enrollment, tertiary (gross), gender parity index (GPI)
- School enrollment, tertiary, female (% gross)
- School enrollment, tertiary, male (% gross)
- Total alcohol consumption per capita (liters of pure alcohol, projected estimates, 15+ years of age)
- Total alcohol consumption per capita, female (liters of pure alcohol, projected estimates, female 15+ years of age)
- Total alcohol consumption per capita, male (liters of pure alcohol, projected estimates, male 15+ years of age)
- Total reserves (% of total external debt)
- Total reserves in months of imports
- Trade (% of GDP)

```
In [5]: # Print statistics for each column one by one
for col in data.columns:
    print(f"\n{' '*80}")
    print(f"Column: {col}")
    print(f"{' '*80}")
    print(f>Data type: {data[col].dtype}")
    print(f>Missing values: {data[col].isna().sum()}")
    print(f>Unique values: {data[col].nunique()}")

    if pd.api.types.is_numeric_dtype(data[col]):
        print(f>Mean: {data[col].mean()}")
        print(f>Std: {data[col].std()}")
        print(f>Min: {data[col].min()}")
        print(f>25%: {data[col].quantile(0.25)}")
        print(f>Median: {data[col].median()}")
        print(f>75%: {data[col].quantile(0.75)}")
        print(f>Max: {data[col].max()}")
    else:
        print("Top values:")
        print(data[col].value_counts(dropna=False).head(10))
```

```
=====
Column: Country Name
=====
```

Data type: object

Missing values: 0

Unique values: 11

Top values:

Country Name

Brunei Darussalam 21

Cambodia 21

Indonesia 21

Lao PDR 21

Malaysia 21

Myanmar 21

Philippines 21

Singapore 21

Thailand 21

Timor-Leste 21

Name: count, dtype: int64

```
=====
Column: Year
=====
```

Data type: int64

Missing values: 0

Unique values: 21

Mean: 2010.0

Std: 6.068450128041075

Min: 2000

25%: 2005.0

Median: 2010.0

75%: 2015.0

Max: 2020

```
=====
Column: Adolescents out of school (% of lower secondary school age)
=====
```

Data type: float64

Missing values: 81

Unique values: 150

Mean: 19.143566189882584

Std: 16.49151226668506

Min: 0.0646251863563981

25%: 9.916988138615144

Median: 14.183316800426201

75%: 24.1658674862409

Max: 84.9968490600586

```
=====
Column: Children out of school (% of primary school age)
=====
```

Data type: float64

Missing values: 75

Unique values: 153

Mean: 6.211913161706693

Std: 5.360247737329246

Min: 0.0
25%: 1.914034962654115
Median: 5.212002085937151
75%: 8.084092413421335
Max: 25.6012496948242

=====
Column: Current account balance (% of GDP)
=====

Data type: float64
Missing values: 8
Unique values: 223
Mean: 12.196330652435568
Std: 40.65240242099289
Min: -32.4953733274264
25%: -2.1685774257797847
Median: 2.65327822085759
75%: 11.579638502362151
Max: 311.746071075818

=====
Column: Current health expenditure (% of GDP)
=====

Data type: float64
Missing values: 3
Unique values: 228
Mean: 3.7437725746929824
Std: 1.3633857196061034
Min: 1.85097206
25%: 2.7443040625000004
Median: 3.487886665
75%: 4.40756214
Max: 8.5323782

=====
Column: Current health expenditure per capita, PPP (current international \$)
=====

Data type: float64
Missing values: 3
Unique values: 228
Mean: 641.8412712517421
Std: 902.636902816964
Min: 20.1420367058147
25%: 133.2366246861425
Median: 255.917697781433
75%: 650.281910371236
Max: 5854.30406536757

=====
Column: Debt service (PPG and IMF only, % of exports of goods, services and primary income)
=====

Data type: float64
Missing values: 69
Unique values: 157
Mean: 5.067103262404221

Std: 5.139557540660075
Min: 0.0
25%: 0.940349798441852
Median: 3.19609377966218
75%: 7.9319453673303
Max: 20.2489854683495

=====
Column: Domestic general government health expenditure (% of general government expenditure)
=====

Data type: float64
Missing values: 3
Unique values: 228
Mean: 6.766840138552631
Std: 3.202196595639124
Min: 1.16742086
25%: 4.5098938925
Median: 6.51266694
75%: 8.462214470000001
Max: 16.28944588

=====
Column: Educational attainment, Doctoral or equivalent, population 25+, female (%) (cumulative)
=====

Data type: float64
Missing values: 208
Unique values: 19
Mean: 0.15759497448765522
Std: 0.3811221383047414
Min: 0.0
25%: 0.02499999944120645
Median: 0.0462067104949552
75%: 0.1400000005960465
Max: 1.86660751453189

=====
Column: Educational attainment, Doctoral or equivalent, population 25+, male (%) (cumulative)
=====

Data type: float64
Missing values: 208
Unique values: 20
Mean: 0.19687158636539745
Std: 0.30133512889354536
Min: 0.0359788062798414
25%: 0.057818743057637
Median: 0.100000001490116
75%: 0.170000001788139
Max: 1.38404682010056

=====
Column: Educational attainment, Doctoral or equivalent, population 25+, total (%) (cumulative)
=====

Data type: float64
Missing values: 205
Unique values: 20
Mean: 0.20500538098789786
Std: 0.3233478767038197
Min: 0.0299999993294477
25%: 0.04950968654426982
Median: 0.08336926346709675
75%: 0.247499991208315
Max: 1.60901415118158

=====
Column: Educational attainment, at least Bachelor's or equivalent, population 25+, female (%) (cumulative)
=====

Data type: float64
Missing values: 175
Unique values: 56
Mean: 14.59886718727864
Std: 8.332236448158385
Min: 0.121030002832413
25%: 8.001707315444948
Median: 14.5501705757295
75%: 19.960062044428348
Max: 31.7299995422363

=====
Column: Educational attainment, at least Bachelor's or equivalent, population 25+, male (%) (cumulative)
=====

Data type: float64
Missing values: 173
Unique values: 58
Mean: 15.166032408547114
Std: 9.113506242586055
Min: 0.107119999825954
25%: 8.717056762527848
Median: 12.444115603309651
75%: 22.147500514984152
Max: 34.3600006103516

=====
Column: Educational attainment, at least Bachelor's or equivalent, population 25+, total (%) (cumulative)
=====

Data type: float64
Missing values: 174
Unique values: 57
Mean: 14.928550896441017
Std: 8.521628833721122
Min: 0.114589996635914
25%: 8.32035087371126
Median: 13.9771413500091
75%: 20.3400001525879
Max: 32.9799995422363

```

=====
Column: Educational attainment, at least Master's or equivalent, population 25+, female (%) (cumulative)
=====
Data type: float64
Missing values: 197
Unique values: 33
Mean: 1.3784849007195759
Std: 1.6208490044829469
Min: 0.180000007152557
25%: 0.368256750507007
Median: 0.56214651756915
75%: 1.889999955892565
Max: 6.27942799439893

=====
Column: Educational attainment, at least Master's or equivalent, population 25+, male (%) (cumulative)
=====
Data type: float64
Missing values: 192
Unique values: 38
Mean: 1.4051875725552871
Std: 1.3735699823438638
Min: 0.159999996423721
25%: 0.6678478198294425
Median: 0.819999992847443
75%: 1.72999995946884
Max: 5.82812150274552

=====
Column: Educational attainment, at least Master's or equivalent, population 25+, total (%) (cumulative)
=====
Data type: float64
Missing values: 192
Unique values: 38
Mean: 1.3572570013222771
Std: 1.4566818338969894
Min: 0.258716366146977
25%: 0.4614705402601505
Median: 0.610000014305115
75%: 1.8449999690055852
Max: 5.28088224770611

=====
Column: Educational attainment, at least completed primary, population 25+ years, total (%) (cumulative)
=====
Data type: float64
Missing values: 140
Unique values: 90
Mean: 72.62145502241756
Std: 17.476949180787226
Min: 22.5100002288818
25%: 65.42733046133284

```

Median: 79.1305519146926
75%: 84.10000228881836
Max: 94.370002746582

=====
Column: Educational attainment, at least completed upper secondary, population 25+,
total (%) (cumulative)
=====

Data type: float64
Missing values: 129
Unique values: 102
Mean: 36.65671433158695
Std: 20.420496635289712
Min: 5.91235625969463
25%: 22.3606688554265
Median: 30.54486530484765
75%: 58.218724289631474
Max: 74.5500030517578

=====
Column: Exports of goods and services (% of GDP)
=====

Data type: float64
Missing values: 25
Unique values: 206
Mean: 64.10867790583457
Std: 50.681533561604134
Min: 2.40901270152234
25%: 30.523741986126698
Median: 56.6250569021147
75%: 68.93149551491065
Max: 228.993770935119

=====
Column: External debt stocks (% of GNI)
=====

Data type: float64
Missing values: 75
Unique values: 156
Mean: 46.121446996198245
Std: 31.53882416095133
Min: 1.76077705580083
25%: 27.9791589442757
Median: 37.40370289748235
75%: 56.63682850373255
Max: 183.229264533177

=====
Column: Foreign direct investment, net inflows (% of GDP)
=====

Data type: float64
Missing values: 4
Unique values: 227
Mean: 4.807567647066827
Std: 5.839506032540802
Min: -16.2053004878706

25%: 1.878538045494385
Median: 3.28081949125174
75%: 5.13843663056025
Max: 29.6706126230568

=====
Column: Foreign direct investment, net outflows (% of GDP)
=====

Data type: float64
Missing values: 61
Unique values: 170
Mean: 2.442743341711479
Std: 4.496263197924893
Min: -3.16536257677267
25%: 0.18464212142225425
Median: 0.7499341924199285
75%: 2.1522864408152476
Max: 22.5941532405914

=====
Column: GDP (current US\$)
=====

Data type: float64
Missing values: 0
Unique values: 231
Mean: 171321456033.49927
Std: 224081622703.46924
Min: 366924276.517996
25%: 12066983849.554
Median: 84307345887.543
75%: 282806729388.0185
Max: 1119099871350.2

=====
Column: GDP growth (annual %)
=====

Data type: float64
Missing values: 0
Unique values: 231
Mean: 5.596433318546239
Std: 5.710658731861131
Min: -9.51829474045297
25%: 3.745992371588315
Median: 5.75141287983195
75%: 7.34688133069772
Max: 58.078100510636

=====
Column: GDP per capita (constant 2015 US\$)
=====

Data type: float64
Missing values: 0
Unique values: 231
Mean: 9796.538179730971
Std: 15501.295010342023
Min: 298.590464510138

25%: 1307.139406705445
Median: 2439.28655744423
75%: 7129.68500346761
Max: 61345.5291820062

=====
Column: GDP per capita growth (annual %)
=====

Data type: float64
Missing values: 0
Unique values: 231
Mean: 4.100097079440565
Std: 6.632685190418749
Min: -13.8014085738351
25%: 2.3067725284314697
Median: 4.23097600258968
75%: 5.846423280574545
Max: 77.0895736636832

=====
Column: Government expenditure on education, total (% of GDP)
=====

Data type: float64
Missing values: 63
Unique values: 168
Mean: 3.572504203531866
Std: 1.8728780073466929
Min: 0.787440001964569
25%: 2.5278206781970702
Median: 3.2071114802706653
75%: 4.03465247154236
Max: 12.9024801254272

=====
Column: Government expenditure on education, total (% of government expenditure)
=====

Data type: float64
Missing values: 63
Unique values: 168
Mean: 15.451074666863763
Std: 5.528077327322119
Min: 5.30208015441895
25%: 10.8681771549496
Median: 15.588424704791201
75%: 19.3446822166443
Max: 31.3717498779297

=====
Column: Gross savings (% of GDP)
=====

Data type: float64
Missing values: 32
Unique values: 199
Mean: 41.967858808221976
Std: 46.93040895080961
Min: 1.39671283713829

25%: 26.207611973977052
Median: 31.9424672317789
75%: 39.910418275189954
Max: 372.975219889793

=====
Column: Immunization, DPT (% of children ages 12-23 months)
=====

Data type: float64
Missing values: 2
Unique values: 46
Mean: 86.27947598253274
Std: 12.791189665450771
Min: 45.0
25%: 80.0
Median: 90.0
75%: 97.0
Max: 99.0

=====
Column: Imports of goods and services (% of GDP)
=====

Data type: float64
Missing values: 25
Unique values: 206
Mean: 68.64663447769529
Std: 43.39910158246875
Min: 15.6410071153504
25%: 38.25707037168692
Median: 59.9376855600607
75%: 73.10495154890526
Max: 208.332943955938

=====
Column: Life expectancy at birth, female (years)
=====

Data type: float64
Missing values: 0
Unique values: 227
Mean: 73.00092640692642
Std: 6.42390115717616
Min: 57.592
25%: 68.07
Median: 72.389
75%: 78.055
Max: 85.9

=====
Column: Life expectancy at birth, male (years)
=====

Data type: float64
Missing values: 0
Unique values: 229
Mean: 67.58781818181818
Std: 5.869634247683669
Min: 55.119

25%: 63.718
Median: 67.296
75%: 72.5085
Max: 81.4

=====
Column: Life expectancy at birth, total (years)
=====

Data type: float64
Missing values: 0
Unique values: 230
Mean: 70.24854851652412
Std: 6.058033016653247
Min: 56.364
25%: 65.90350000000001
Median: 69.733
75%: 74.817
Max: 83.5951219512195

=====
Column: Literacy rate, youth (ages 15-24), gender parity index (GPI)
=====

Data type: float64
Missing values: 153
Unique values: 19
Mean: 0.9865097778052843
Std: 0.04686080033169213
Min: 0.67330002784729
25%: 0.990000009536743
Median: 1.0
75%: 1.0
Max: 1.02999997138977

=====
Column: Literacy rate, youth total (% of people ages 15-24)
=====

Data type: float64
Missing values: 153
Unique values: 71
Mean: 95.29689664887064
Std: 5.990584328189281
Min: 77.96
25%: 94.63249778747557
Median: 98.0150032043457
75%: 99.4675006866455
Max: 100.0

=====
Column: Mortality caused by road traffic injury (per 100,000 population)
=====

Data type: float64
Missing values: 11
Unique values: 146
Mean: 17.58409090909091
Std: 8.583214562688081
Min: 2.1

25%: 12.174999999999999
Median: 15.5
75%: 23.25
Max: 39.3

=====
Column: Mortality from CVD, cancer, diabetes or CRD between exact ages 30 and 70 (%)
=====

Data type: float64
Missing values: 0
Unique values: 131
Mean: 21.66796536796537
Std: 5.659498727120068
Min: 8.6
25%: 17.9
Median: 21.4
75%: 25.65
Max: 34.3

=====
Column: Mortality from CVD, cancer, diabetes or CRD between exact ages 30 and 70, female (%)
=====

Data type: float64
Missing values: 0
Unique values: 135
Mean: 18.272294372294372
Std: 5.69966539215535
Min: 6.4
25%: 14.2
Median: 18.2
75%: 22.45
Max: 31.2

=====
Column: Mortality from CVD, cancer, diabetes or CRD between exact ages 30 and 70, male (%)
=====

Data type: float64
Missing values: 0
Unique values: 144
Mean: 25.372727272727275
Std: 6.434748798116494
Min: 10.8
25%: 19.9
Median: 27.6
75%: 30.1
Max: 38.1

=====
Column: Mortality rate, infant (per 1,000 live births)
=====

Data type: float64
Missing values: 0
Unique values: 171
Mean: 28.545887445887445

Std: 22.72629329491299
Min: 1.9
25%: 8.5
Median: 22.4
75%: 43.65
Max: 86.8

=====
Column: Mortality rate, under-5 (per 1,000 live births)
=====

Data type: float64
Missing values: 0
Unique values: 184
Mean: 35.36666666666667
Std: 28.190417222673485
Min: 2.3
25%: 10.0
Median: 27.9
75%: 53.8
Max: 110.5

=====
Column: School enrollment, primary (% net)
=====

Data type: float64
Missing values: 101
Unique values: 130
Mean: 92.80185215384616
Std: 4.974103454213958
Min: 75.377
25%: 89.984765
Median: 93.505345
75%: 96.235775
Max: 99.8924

=====
Column: School enrollment, primary (gross), gender parity index (GPI)
=====

Data type: float64
Missing values: 32
Unique values: 198
Mean: 0.9706394202146099
Std: 0.0361866708590609
Min: 0.846400022506714
25%: 0.9549171030521395
Median: 0.976999998092651
75%: 0.9966524243354795
Max: 1.03632998466492

=====
Column: School enrollment, primary, female (% net)
=====

Data type: float64
Missing values: 133
Unique values: 98
Mean: 91.07947622448981

Std: 5.7614187074490015
Min: 71.90024
25%: 89.0134175
Median: 91.49965
75%: 94.6948325
Max: 99.8099

=====
Column: School enrollment, primary, male (% net)
=====

Data type: float64
Missing values: 133
Unique values: 98
Mean: 92.35072959183674
Std: 4.547349669887121
Min: 78.76963
25%: 90.1763725
Median: 92.511015
75%: 95.7681125
Max: 99.97004

=====
Column: School enrollment, secondary (% net)
=====

Data type: float64
Missing values: 108
Unique values: 123
Mean: 59.921059105691064
Std: 19.784970332321226
Min: 15.61553
25%: 43.52194
Median: 60.0122
75%: 75.37781
Max: 99.78462

=====
Column: School enrollment, secondary (gross), gender parity index (GPI)
=====

Data type: float64
Missing values: 71
Unique values: 160
Mean: 0.99242487475276
Std: 0.11508622257445264
Min: 0.544290006160736
25%: 0.9730500131845475
Median: 1.01191961765289
75%: 1.0758775174617725
Max: 1.12862002849579

=====
Column: School enrollment, secondary, female (% net)
=====

Data type: float64
Missing values: 109
Unique values: 122
Mean: 61.012772131147536

Std: 21.455440211821337
Min: 11.18099
25%: 42.785255
Median: 64.62856500000001
75%: 78.1930975
Max: 99.79816

=====
Column: School enrollment, secondary, male (% net)
=====

Data type: float64
Missing values: 109
Unique values: 122
Mean: 59.12509762295082
Std: 18.4919135718917
Min: 19.98272
25%: 42.8265225
Median: 57.29297
75%: 73.1158475
Max: 99.7717

=====
Column: School enrollment, tertiary (% gross)
=====

Data type: float64
Missing values: 57
Unique values: 174
Mean: 27.571496651968065
Std: 18.84424074539369
Min: 1.86557996273041
25%: 14.7145049571991
Median: 26.099720808865747
75%: 35.797486350749175
Max: 101.99681244982

=====
Column: School enrollment, tertiary (gross), gender parity index (GPI)
=====

Data type: float64
Missing values: 61
Unique values: 169
Mean: 1.0815595166871816
Std: 0.2836717070780884
Min: 0.311450004577637
25%: 0.9293948722237337
Median: 1.13539179794344
75%: 1.265412539243695
Max: 1.55392003059387

=====
Column: School enrollment, tertiary, female (% gross)
=====

Data type: float64
Missing values: 61
Unique values: 169
Mean: 30.45825046159326

Std: 20.988258789103625
Min: 0.903339982032776
25%: 14.7983295844916
Median: 29.450883781018298
75%: 40.85187873626902
Max: 105.310905467206

=====
Column: School enrollment, tertiary, male (% gross)
=====

Data type: float64
Missing values: 61
Unique values: 170
Mean: 25.509179239768503
Std: 17.400060062994612
Min: 2.90040993690491
25%: 13.670475157177751
Median: 23.139897633700002
75%: 31.401597795228252
Max: 98.9005470222827

=====
Column: Total alcohol consumption per capita (liters of pure alcohol, projected estimates, 15+ years of age)
=====

Data type: float64
Missing values: 0
Unique values: 167
Mean: 3.6489047619047614
Std: 3.5881763661181054
Min: 0.055
25%: 0.51
Median: 1.91
75%: 6.945
Max: 11.52

=====
Column: Total alcohol consumption per capita, female (liters of pure alcohol, projected estimates, female 15+ years of age)
=====

Data type: float64
Missing values: 0
Unique values: 136
Mean: 1.4572510822510822
Std: 1.474365050420349
Min: 0.018
25%: 0.17
Median: 0.8
75%: 2.795
Max: 4.85

=====
Column: Total alcohol consumption per capita, male (liters of pure alcohol, projected estimates, male 15+ years of age)
=====

Data type: float64

Missing values: 0
Unique values: 175
Mean: 5.8999523809523815
Std: 5.780537104756071
Min: 0.089
25%: 0.815
Median: 2.94
75%: 11.335
Max: 18.15

=====
Column: Total reserves (% of total external debt)
=====

Data type: float64
Missing values: 75
Unique values: 156
Mean: 81.98028697767651
Std: 127.44223707347015
Min: 4.41971596348517
25%: 27.1300748192998
Median: 46.1395068666154
75%: 99.735815003195
Max: 1162.02765610153

=====
Column: Total reserves in months of imports
=====

Data type: float64
Missing values: 8
Unique values: 223
Mean: 5.273209225062216
Std: 2.330911200151504
Min: 1.30881642367651
25%: 3.240891528971175
Median: 5.16536886202672
75%: 6.54181941111122
Max: 12.2854455443743

=====
Column: Trade (% of GDP)
=====

Data type: float64
Missing values: 25
Unique values: 206
Mean: 132.75531238352988
Std: 87.51842993040722
Min: 32.9721754002427
25%: 80.89075766388208
Median: 116.2147493886565
75%: 138.31327419640925
Max: 437.326714891057

V. Preprocessing data

Data preprocessing is an important step in the analysis workflow because raw data is often incomplete, inconsistent, or not yet structured in a form suitable for analysis and visualization. In this project, preprocessing helps ensure that the dataset is clean, readable, and ready for statistical exploration.

Handling missing values

- For each country, missing numeric values are then filled over time using linear interpolation, followed by forward fill and backward fill to handle remaining gaps at the beginning or end of the series.
- This makes the dataset more complete and reliable for later analysis and visualization.

```
In [6]: data = data.sort_values(MERGE_KEYS).reset_index(drop=True)

numeric_cols = [
    col for col in data.columns
    if col not in MERGE_KEYS and pd.api.types.is_numeric_dtype(data[col])
]

# Fill missing indicator values within each country across time.
data[numeric_cols] = data.groupby("Country Name")[numeric_cols].transform(
    lambda values: values.interpolate().ffill().bfill()
)

In [7]: data_imputed = data.copy()

n_years = data_imputed["Year"].nunique()
remaining_missing = data_imputed[numeric_cols].isna().sum()

# Track which values were imputed
imputed_flag = pd.DataFrame(False, index=data_imputed.index, columns=numeric_cols)

# 1. Deterministic fill: Trade = Exports + Imports
trade_col = "Trade (% of GDP)"
exports_col = "Exports of goods and services (% of GDP)"
imports_col = "Imports of goods and services (% of GDP)"

if {trade_col, exports_col, imports_col}.issubset(data_imputed.columns):
    mask = (
        data_imputed[trade_col].isna()
        & data_imputed[exports_col].notna()
        & data_imputed[imports_col].notna()
    )
    data_imputed.loc[mask, trade_col] = (
        data_imputed.loc[mask, exports_col] + data_imputed.loc[mask, imports_col]
    )
    imputed_flag.loc[mask, trade_col] = True

# 2. Safe fill: fill columns missing for exactly one country across all years (21 v
one_country_missing_cols = remaining_missing[remaining_missing == n_years].index.to
```

```

for col in one_country_missing_cols:
    year_median = data_imputed.groupby("Year")[col].transform("median")
    mask = data_imputed[col].isna() & year_median.notna()
    data_imputed.loc[mask, col] = year_median[mask]
    imputed_flag.loc[mask, col] = True

# 3. Check result
remaining_final = data_imputed[numeric_cols].isna().sum()
print("Remaining missing values after imputation:")
print(remaining_final[remaining_final > 0])

# 4. Count how many values were imputed per column
print("\nNumber of imputed values per column:")
print(imputed_flag.sum()[imputed_flag.sum() > 0])

```

Remaining missing values after imputation:

```

Debt service (PPG and IMF only, % of exports of goods, services and primary income)
63
Educational attainment, Doctoral or equivalent, population 25+, female (%) (cumulative)
42
Educational attainment, Doctoral or equivalent, population 25+, male (%) (cumulative)
42
Educational attainment, Doctoral or equivalent, population 25+, total (%) (cumulative)
42
Educational attainment, at least Master's or equivalent, population 25+, female (%) (cumulative)
42
Educational attainment, at least Master's or equivalent, population 25+, male (%) (cumulative)
42
Educational attainment, at least Master's or equivalent, population 25+, total (%) (cumulative)
42
External debt stocks (% of GNI)
63
School enrollment, primary, female (% net)
63
School enrollment, primary, male (% net)
63
Total reserves (% of total external debt)
63
dtype: int64

```

Number of imputed values per column:

```

Exports of goods and services (% of GDP)                21
Foreign direct investment, net outflows (% of GDP)       21
Gross savings (% of GDP)                                21
Imports of goods and services (% of GDP)                 21
School enrollment, secondary (% net)                     21
School enrollment, secondary (gross), gender parity index (GPI) 21
School enrollment, secondary, female (% net)             21
School enrollment, secondary, male (% net)               21
Trade (% of GDP)                                         21
dtype: int64

```

```

In [8]: def compress_years(years):
        years = sorted(set(int(year) for year in years))
        if not years:
            return ""

```

```

ranges = []
start = prev = years[0]

for year in years[1:]:
    if year == prev + 1:
        prev = year
    else:
        ranges.append(f"{start}-{prev}" if start != prev else str(start))
        start = prev = year

ranges.append(f"{start}-{prev}" if start != prev else str(start))
return ", ".join(ranges)

def describe_country_years(rows):
    if rows.empty:
        return "None"

    parts = []
    for country, group in rows.groupby("Country Name", sort=True):
        parts.append(f"{country}: {compress_years(group['Year'].tolist())}")
    return "; ".join(parts)

OBJ1_COLS = [
    "School enrollment, primary (% net)",
    "Children out of school (% of primary school age)",
    "School enrollment, secondary (% net)",
    "Adolescents out of school (% of lower secondary school age)",
]

OBJ2_COLS = [
    "Government expenditure on education, total (% of GDP)",
    "Government expenditure on education, total (% of government expenditure)",
    "School enrollment, primary (% net)",
    "Children out of school (% of primary school age)",
    "School enrollment, secondary (% net)",
    "Adolescents out of school (% of lower secondary school age)",
]

OBJ3_COLS = [
    "School enrollment, primary, female (% net)",
    "School enrollment, primary, male (% net)",
    "School enrollment, secondary, female (% net)",
    "School enrollment, secondary, male (% net)",
    "School enrollment, tertiary, female (% gross)",
    "School enrollment, tertiary, male (% gross)",
    "School enrollment, primary (gross), gender parity index (GPI)",
    "School enrollment, secondary (gross), gender parity index (GPI)",
    "School enrollment, tertiary (gross), gender parity index (GPI)",
]

OBJ4_COLS = [
    "Literacy rate, youth total (% of people ages 15-24)",
    "Literacy rate, youth (ages 15-24), gender parity index (GPI)",
    "School enrollment, secondary (% net)",

```

```

    "School enrollment, tertiary (% gross)",
]

OBJ5_COLS = [
    "Educational attainment, at least completed primary, population 25+ years, total",
    "Educational attainment, at least completed upper secondary, population 25+, total",
    "Educational attainment, at least Bachelor's or equivalent, population 25+, total",
    "Educational attainment, at least Bachelor's or equivalent, population 25+, female",
    "Educational attainment, at least Bachelor's or equivalent, population 25+, male",
    "Educational attainment, at least Master's or equivalent, population 25+, total",
    "Educational attainment, at least Master's or equivalent, population 25+, female",
    "Educational attainment, at least Master's or equivalent, population 25+, male",
    "Educational attainment, Doctoral or equivalent, population 25+, total (%) (cumulative)",
    "Educational attainment, Doctoral or equivalent, population 25+, female (%) (cumulative)",
    "Educational attainment, Doctoral or equivalent, population 25+, male (%) (cumulative)",
]

OBJ6_COLS = [
    "Life expectancy at birth, total (years)"]

OBJ7_COLS = [
    "Life expectancy at birth, total (years)",
    "GDP (current US$)",
    "GDP growth (annual %)",
    "Current health expenditure per capita, PPP (current international $)",
    "Current health expenditure (% of GDP)",
    "Domestic general government health expenditure (% of general government expenditure)",
]

OBJ8_COLS = [
    "Mortality caused by road traffic injury (per 100,000 population)",
    "Total alcohol consumption per capita (liters of pure alcohol, projected estimate)",
    "Mortality from CVD, cancer, diabetes or CRD between exact ages 30 and 70 (%)"
]

OBJ9_COLS = [
    "Mortality rate, under-5 (per 1,000 live births)",
    "Mortality rate, infant (per 1,000 live births)",
    "Immunization, HepB3 (% of one-year-old children)",
    "Current health expenditure per capita, PPP (current international $)"
]

OBJ10_COLS = [
    "Life expectancy at birth, male (years)",
    "Life expectancy at birth, female (years)",
    "Mortality from CVD, cancer, diabetes or CRD between exact ages 30 and 70, male",
    "Mortality from CVD, cancer, diabetes or CRD between exact ages 30 and 70, female",
    "Total alcohol consumption per capita, male (liters of pure alcohol, projected estimate)",
    "Total alcohol consumption per capita, female (liters of pure alcohol, projected estimate)"
]

OBJ11_COLS = [
    "GDP per capita (constant 2015 US$)",
    "GDP per capita growth (annual %)"
]

```

```

OBJ12_COLS = [
    "Trade (% of GDP)",
    "Exports of goods and services (% of GDP)",
    "Imports of goods and services (% of GDP)"
]

OBJ13_COLS = [
    "Foreign direct investment, net inflows (% of GDP)",
    "GDP per capita growth (annual %)",
    "Exports of goods and services (% of GDP)"
]

OBJ14_COLS = [
    "External debt stocks (% of GNI)",
    "Debt service (% of exports)",
    "Total reserves (% of total external debt)",
    "Total reserves in months of imports"
]

OBJ15_COLS = [
    "Gross savings (% of GDP)",
    "Foreign direct investment, net inflows (% of GDP)",
    "Current account balance (% of GDP)"
]

OBJECTIVE_VARS = {
    "Objective 1 - Enrollment and Out-of-School": OBJ1_COLS,
    "Objective 2 - Education Spending and Access": OBJ2_COLS,
    "Objective 3 - Gender Parity in Enrollment": OBJ3_COLS,
    "Objective 4 - Youth Literacy and Higher-Level Participation": OBJ4_COLS,
    "Objective 5 - Educational Attainment": OBJ5_COLS,
    "Objective 6 - Health and Economic Indicators": OBJ6_COLS,
    "Objective 7 - Health and Economic Indicators": OBJ7_COLS,
    "Objective 8 - Health Risk Factors and Outcomes": OBJ8_COLS,
    "Objective 9 - Health Indicators": OBJ9_COLS,
    "Objective 10 - Gender-Specific Health Indicators": OBJ10_COLS,
    "Objective 11 - Economic Indicators": OBJ11_COLS,
    "Objective 12 - Trade and Economic Integration": OBJ12_COLS,
    "Objective 13 - Foreign Investment and Economic Growth": OBJ13_COLS,
    "Objective 14 - Debt Sustainability": OBJ14_COLS,
    "Objective 15 - Domestic Savings and Economic Vulnerability": OBJ15_COLS
}

analysis_ready_by_objective = {}
objective_cautions = {}
objective_missing_details = {}
objective_summary_rows = []

for objective_name, cols in OBJECTIVE_VARS.items():
    cols = [col for col in cols if col in data_imputed.columns]

    if not cols:
        analysis_ready_by_objective[objective_name] = data_imputed[["Country Name",
        objective_missing_details[objective_name] = {}
        objective_cautions[objective_name] = (
            f"Caution for {objective_name}: no valid indicators were found in the d
        )

```

```

        objective_summary_rows.append(
            {
                "Objective": objective_name,
                "Indicators used": 0,
                "Countries covered": data_imputed["Country Name"].nunique(),
                "Indicators imputed": 0,
                "Indicators still missing": 0,
            }
        )
        continue

    objective_df = data_imputed[["Country Name", "Year"] + cols].copy()
    analysis_ready_by_objective[objective_name] = objective_df

    imputed_cols = [col for col in cols if imputed_flag[col].any()]
    remaining_cols = [col for col in cols if objective_df[col].isna().any()]

    objective_missing_details[objective_name] = {
        col: objective_df.loc[objective_df[col].isna(), ["Country Name", "Year"]].c
        for col in remaining_cols
    }

    caution_lines = [f"Caution for {objective_name}:"]

    caution_lines = [f"Caution for {objective_name}:"]

    if imputed_cols:
        caution_lines.append("This objective uses imputed values for single-country
        for col in imputed_cols:
            rows = objective_df.loc[imputed_flag.loc[objective_df.index, col], ["Co
            caution_lines.append(f"- {col}: {describe_country_years(rows)}")
    else:
        caution_lines.append("No imputation was applied to indicators in this objec

    if remaining_cols:
        caution_lines.append(
            "The following indicators still contain missing values and should be in
            "when a related country appears in the analysis:"
        )
        for col in remaining_cols:
            rows = objective_missing_details[objective_name][col]
            caution_lines.append(f"- {col}: {describe_country_years(rows)}")
    else:
        caution_lines.append("No remaining missing values in this objective after s

    objective_cautions[objective_name] = "\n".join(caution_lines)

    objective_summary_rows.append(
        {
            "Objective": objective_name,
            "Indicators used": len(cols),
            "Countries covered": objective_df["Country Name"].nunique(),
            "Indicators imputed": len(imputed_cols),
            "Indicators still missing": len(remaining_cols),
        }
    )

```

```
)  
  
objective_summary = pd.DataFrame(objective_summary_rows)
```

```
In [9]: # Save the cleaned and merged dataset to a new CSV file  
data_imputed.to_csv("../preprocessed_dataset/SoutheastAsianData.csv", index=False)  
print(f"Cleaned and merged dataset saved to preprocessed_dataset/SoutheastAsianData.csv")
```

Cleaned and merged dataset saved to preprocessed_dataset/SoutheastAsianData.csv

```
In [10]: print("Objective summary:")  
print(objective_summary.to_string(index=False))  
  
for objective_name, caution_text in objective_cautions.items():  
    print("\n" + "=" * 100)  
    print(caution_text)
```

Objective summary:

	Objective	Indicators used	Countr
ies covered	Indicators imputed	Indicators still missing	
	Objective 1 - Enrollment and Out-of-School		4
11	1	0	
	Objective 2 - Education Spending and Access		6
11	1	0	
	Objective 3 - Gender Parity in Enrollment		9
11	3	2	
	Objective 4 - Youth Literacy and Higher-Level Participation		4
11	1	0	
	Objective 5 - Educational Attainment		11
11	0	6	
	Objective 6 - Health and Economic Indicators		1
11	0	0	
	Objective 7 - Health and Economic Indicators		6
11	0	0	
	Objective 8 - Health Risk Factors and Outcomes		3
11	0	0	
	Objective 9 - Health Indicators		3
11	0	0	
	Objective 10 - Gender-Specific Health Indicators		6
11	0	0	
	Objective 11 - Economic Indicators		2
11	0	0	
	Objective 12 - Trade and Economic Integration		3
11	3	0	
	Objective 13 - Foreign Investment and Economic Growth		3
11	1	0	
	Objective 14 - Debt Sustainability		3
11	0	2	
	Objective 15 - Domestic Savings and Economic Vulnerability		3
11	1	0	

Caution for Objective 1 - Enrollment and Out-of-School:

This objective uses imputed values for single-country full-series gaps:

- School enrollment, secondary (% net): Viet Nam: 2000-2020

No remaining missing values in this objective after safe imputation.

Caution for Objective 2 - Education Spending and Access:

This objective uses imputed values for single-country full-series gaps:

- School enrollment, secondary (% net): Viet Nam: 2000-2020

No remaining missing values in this objective after safe imputation.

Caution for Objective 3 - Gender Parity in Enrollment:

This objective uses imputed values for single-country full-series gaps:

- School enrollment, secondary, female (% net): Viet Nam: 2000-2020

- School enrollment, secondary, male (% net): Viet Nam: 2000-2020

- School enrollment, secondary (gross), gender parity index (GPI): Viet Nam: 2000-2020

The following indicators still contain missing values and should be interpreted carefully when a related country appears in the analysis:

- School enrollment, primary, female (% net): Brunei Darussalam: 2000-2020; Singapore: 2000-2020; Viet Nam: 2000-2020
- School enrollment, primary, male (% net): Brunei Darussalam: 2000-2020; Singapore: 2000-2020; Viet Nam: 2000-2020

=====

Caution for Objective 4 - Youth Literacy and Higher-Level Participation:

This objective uses imputed values for single-country full-series gaps:

- School enrollment, secondary (% net): Viet Nam: 2000-2020

No remaining missing values in this objective after safe imputation.

=====

Caution for Objective 5 - Educational Attainment:

No imputation was applied to indicators in this objective.

The following indicators still contain missing values and should be interpreted carefully when a related country appears in the analysis:

- Educational attainment, at least Master's or equivalent, population 25+, total (%) (cumulative): Singapore: 2000-2020; Timor-Leste: 2000-2020
- Educational attainment, at least Master's or equivalent, population 25+, female (%) (cumulative): Singapore: 2000-2020; Timor-Leste: 2000-2020
- Educational attainment, at least Master's or equivalent, population 25+, male (%) (cumulative): Singapore: 2000-2020; Timor-Leste: 2000-2020
- Educational attainment, Doctoral or equivalent, population 25+, total (%) (cumulative): Singapore: 2000-2020; Timor-Leste: 2000-2020
- Educational attainment, Doctoral or equivalent, population 25+, female (%) (cumulative): Singapore: 2000-2020; Timor-Leste: 2000-2020
- Educational attainment, Doctoral or equivalent, population 25+, male (%) (cumulative): Singapore: 2000-2020; Timor-Leste: 2000-2020

=====

Caution for Objective 6 - Health and Economic Indicators:

No imputation was applied to indicators in this objective.

No remaining missing values in this objective after safe imputation.

=====

Caution for Objective 7 - Health and Economic Indicators:

No imputation was applied to indicators in this objective.

No remaining missing values in this objective after safe imputation.

=====

Caution for Objective 8 - Health Risk Factors and Outcomes:

No imputation was applied to indicators in this objective.

No remaining missing values in this objective after safe imputation.

=====

Caution for Objective 9 - Health Indicators:

No imputation was applied to indicators in this objective.

No remaining missing values in this objective after safe imputation.

```

=====
=====
Caution for Objective 10 - Gender-Specific Health Indicators:
No imputation was applied to indicators in this objective.
No remaining missing values in this objective after safe imputation.

=====
=====
Caution for Objective 11 - Economic Indicators:
No imputation was applied to indicators in this objective.
No remaining missing values in this objective after safe imputation.

=====
=====
Caution for Objective 12 - Trade and Economic Integration:
This objective uses imputed values for single-country full-series gaps:
- Trade (% of GDP): Myanmar: 2000-2020
- Exports of goods and services (% of GDP): Myanmar: 2000-2020
- Imports of goods and services (% of GDP): Myanmar: 2000-2020
No remaining missing values in this objective after safe imputation.

=====
=====
Caution for Objective 13 - Foreign Investment and Economic Growth:
This objective uses imputed values for single-country full-series gaps:
- Exports of goods and services (% of GDP): Myanmar: 2000-2020
No remaining missing values in this objective after safe imputation.

=====
=====
Caution for Objective 14 - Debt Sustainability:
No imputation was applied to indicators in this objective.
The following indicators still contain missing values and should be interpreted care
fully when a related country appears in the analysis:
- External debt stocks (% of GNI): Brunei Darussalam: 2000-2020; Malaysia: 2000-202
0; Singapore: 2000-2020
- Total reserves (% of total external debt): Brunei Darussalam: 2000-2020; Malaysia:
2000-2020; Singapore: 2000-2020

=====
=====
Caution for Objective 15 - Domestic Savings and Economic Vulnerability:
This objective uses imputed values for single-country full-series gaps:
- Gross savings (% of GDP): Myanmar: 2000-2020
No remaining missing values in this objective after safe imputation.

```